

JAVASCRIPT

Arrays, Objects & Iteration Methods

push • map • filter • reduce • spread • destructuring

ROADMAP

Three parts, all code-first

1	**Arrays & Their Functions**	push, pop, unshift, indexOf, slice vs splice, concat vs spread, flat, isArray, from.
2	**Objects & Their Functions**	Dot vs bracket access, freeze, merging with assign/spread, and destructuring.
3	**forEach, map, filter, reduce**	The four methods every array-processing pattern in JavaScript builds on.

Every slide pairs the exact code with its printed output and a short explanation of why.

1

Arrays & Their Functions

Creating, growing, shrinking, slicing and reshaping arrays.

Creating arrays — array literal vs. new Array()

CODE

```
const A12 = [23, 34, [45, 67], 76,
             [12, 17, 91], 3];
// dynamic array — mixes numbers & nested
arrays

const A = new Array(1, 2, 3, 4);
console.log(A12);
console.log(A);
```

OUTPUT & EXPLANATION

OUTPUT

```
[23, 34, [45, 67], 76, [12, 17, 91], 3]
[1, 2, 3, 4]
```

WHAT'S HAPPENING

JavaScript arrays don't need every item to be the same type — A12 mixes numbers with nested arrays. new Array(…) is just another way to build one from a list of values.

push() & pop() — add and remove from the end

CODE

```
const C = new Array(34, 56, 78);

C.push(73);
C.push(7321);
console.log(C);

C.pop();
console.log(C);
```

OUTPUT & EXPLANATION

OUTPUT

```
[34, 56, 78, 73, 7321]
[34, 56, 78, 73]
```

WHAT'S HAPPENING

push() adds one or more items to the end of the array and returns the new length. pop() removes just the last item — here 7321 is dropped again.

unshift() & indexOf() — add to the start, find a position

CODE

```
C.unshift(91);
console.log(C);

console.log(C.indexOf(78));
```

OUTPUT & EXPLANATION

OUTPUT

```
[91, 34, 56, 78, 73]
2
```

WHAT'S HAPPENING

unshift() inserts at index 0, shifting every other item one place to the right. indexOf() then scans left to right and returns the position of the first match — 78 sits at index 2.

slice() vs. splice() — copy a portion vs. cut it out

CODE

```
const A = [1,2,3,4,5,6,7,8,9];
const B = A.slice(1, 3);
console.log(B);
console.log(A);

const A1 = [1,2,3,4,5,6,7,8,9];
const C = A1.splice(1, 3);
console.log(C);
console.log(A1);
```

OUTPUT & EXPLANATION

OUTPUT

```
[2, 3]
[1,2,3,4,5,6,7,8,9]
[2, 3, 4]
[1, 5, 6, 7, 8, 9]
```

WHAT'S HAPPENING

slice(1,3) returns a new copy of items from index 1 up to (not including) 3 — the original array A is untouched. splice(1,3) instead removes 3 items starting at index 1 directly from A1, mutating it.

concat() vs. spread — two ways to combine arrays

CODE

```
const A = [34, 65, 91, 78, 90];
const B = [21, 3443, 87, 1, 2, 3];

const C = A.concat(B);
const D = [...A, ...B];

console.log(C);
console.log(D);
```

OUTPUT & EXPLANATION

OUTPUT

```
[34,65,91,78,90,21,3443,87,1,2,3]
[34,65,91,78,90,21,3443,87,1,2,3]
```

WHAT'S HAPPENING

concat() and the spread operator (...) both produce the same brand-new merged array without touching A or B — spread is just the more modern, more flexible syntax (it also works inside object and array literals).

flat(Infinity) — flattening nested arrays

CODE

```
const A = [2, 4, 6, [2,7], [2,7,[9,1]], 45];

const B = A.flat(Infinity);
console.log(B);
```

OUTPUT & EXPLANATION

OUTPUT

```
[2, 4, 6, 2, 7, 2, 7, 9, 1, 45]
```

WHAT'S HAPPENING

flat() pulls nested array items up to the top level. Passing a depth of 1 (the default) only unwraps one level — passing Infinity keeps unwrapping no matter how deeply the arrays are nested.

Array.isArray() & Array.from() — checking and converting

CODE

```
console.log(Array.isArray("Gaurav"));

console.log(Array.from("Gaurav"));
```

OUTPUT & EXPLANATION

OUTPUT

```
false
['G','a','u','r','a','v']
```

WHAT'S HAPPENING

A string is not an array, so isArray() correctly returns false. Array.from() can build a real array out of anything “iterable” though — a string included — splitting it into one array item per character.

2

Objects & Their Functions

Reading, overwriting, merging, locking and unpacking key–value data.

Dot notation vs. bracket notation

CODE

```
const A = {
  1: "Gaurav",
  name: "MERN",
  roll: "007"
};

console.log(A[1]);
console.log(A["name"]);
console.log(A.name);
console.log(A.roll);
```

OUTPUT & EXPLANATION

OUTPUT

```
"Gaurav"
"MERN"
"MERN"
"007"
```

WHAT'S HAPPENING

A[1] and A["name"] both use bracket notation — required for numeric or dynamic keys. A.name is dot notation, only valid for plain identifier keys. Both reach the exact same value.

Overwriting a property, then locking it with Object.freeze()

CODE

```
A.roll = "001";
// overwrite — works fine
console.log(A.roll);

Object.freeze(A);
A.roll = "003";
// silently ignored
console.log(A.roll);
```

OUTPUT & EXPLANATION

OUTPUT

```
"001"
"001"
```

WHAT'S HAPPENING

Objects are mutable by default, so A.roll = "001" simply overwrites the old value. Object.freeze() locks the object though — every write attempt after that (like setting "003") is silently ignored, and roll stays "001".

Merging objects — Object.assign() vs. spread

CODE

```
const A = { 1: "Gaurav", name: "MERN",
            roll: "007" };
const B = { name1: "Raj", location: "Noida" };

const C = { ...A, ...B };
const D = Object.assign({}, A, B);

console.log(C);
```

OUTPUT & EXPLANATION

OUTPUT

```
{ 1:"Gaurav", name:"MERN", roll:"007",
  name1:"Raj", location:"Noida" }
```

WHAT'S HAPPENING

Both C and D combine every key from A and B into one brand-new object, leaving A and B unchanged. If a key existed in both, the later source (B) would win — spread and Object.assign() follow the exact same merge order.

Object destructuring — unpacking keys into variables

CODE

```
const ABC = {
  A: "ghg", B: "hgjgj", C: "fhgfhgj"
};

const { A, B, C } = ABC;

console.log(A);
```

OUTPUT & EXPLANATION

OUTPUT

```
"ghg"
```

WHAT'S HAPPENING

Destructuring pulls ABC's properties straight out into standalone variables named A, B and C — no repeated ABC.something needed afterward. The variable names must match the object's key names.

3

forEach, map, filter, reduce

The four methods every array-processing pattern in JavaScript builds on.

forEach() — run code per item, but returns nothing

CODE

```
const B = [34, 45, 87, 91];

const C = B.forEach((item) => {
  console.log(item);
});

console.log(C);
```

OUTPUT & EXPLANATION

OUTPUT

```
34
45
87
91
undefined
```

WHAT'S HAPPENING

forEach() calls the callback once per item purely for its side effects (like logging) — it never builds or returns a new array. Whatever you assign its result to (C here) will always be undefined.

filter() — keep only the items that pass a test

CODE

```
const A = [1, 2, 3, 4, 5, 6, 7];

const D = A.filter((i) => i > 3);

console.log(D);
```

OUTPUT & EXPLANATION

OUTPUT

```
[4, 5, 6, 7]
```

WHAT'S HAPPENING

filter() runs the callback for every item and keeps only the ones where it returns true — here, everything greater than 3. It always returns a new array, which can be shorter than the original (or even empty).

map() — transform every item, one-to-one

CODE

```
const A = [1, 2, 3, 4, 5, 6, 7];

const F = A.map((i) => i * 2);

console.log(F);
```

OUTPUT & EXPLANATION

OUTPUT

```
[2, 4, 6, 8, 10, 12, 14]
```

WHAT'S HAPPENING

map() also returns a new array, but instead of filtering items out, it replaces each one with whatever the callback returns — the result is always the exact same length as the original array.

reduce() — collapse the whole array into one value

CODE

```
const A = [1, 2, 3, 4, 5, 6, 7, 8];

const B = A.reduce((acc, curval) => {
  return acc * curval;
}, 3);

console.log(B);
```

OUTPUT & EXPLANATION

OUTPUT

```
120960
```

WHAT'S HAPPENING

reduce() carries an accumulator (acc) through every item, updating it with whatever the callback returns. Starting from 3 and multiplying by every item (1×2×3…×8 = 40320) gives 3 × 40320 = 120960.

WRAP-UP

Three building blocks, one mental model

1 **Arrays**

Ordered lists you grow, shrink, slice and reshape — push/pop/unshift at the ends, slice/splice in the middle, spread to combine.

2 **Objects**

Key–value data you read with dot or bracket notation, merge with spread or Object.assign, and unpack with destructuring.

3 **Iteration**

forEach for side effects, filter to keep a subset, map to transform, reduce to collapse it all into a single value.

Thank You !